DBMS Mini Project

Personal Expense Splitting App

What was built, the software used, and why

Database: **MongoDB 8** | Language: **JavaScript (mongosh)** | Platform: **macOS**

This document explains, in plain language, every part of the project: the problem it solves, how the database is designed, the queries it answers, every piece of software involved and the reason each one was chosen, and the exact steps to run it.

Section	Contents
1	Project overview
2	Everything that was done, step by step
3	Software used and why
4	Database design
5	The four key queries and their results
6	Project files
7	How to run it on a Mac
8	Problems we hit and how they were fixed
9	Conclusion and future scope

1. Project overview

Friends, roommates and families often share costs: rent, bills, trips and meals. It quickly becomes confusing to remember who paid, who owes, and how much. This project builds the **database** behind an app that keeps track of all of it.

What the database must store

- **Users** - the people using the app.
- **Groups** - e.g. a flat or a trip; a user can be in many groups.
- **Expenses** - who paid, how much, and for what.
- **Expense Participants** - who shares each expense and their share.
- **Settlements / Payments** - money paid back to clear a debt.

What the database must answer

- **Query 1:** all expenses and total spending of a selected group.
- **Query 2:** how much each member owes or is owed.
- **Query 3:** the expense history and contributions of one user.
- **Query 4:** unsettled balances, and recording settlement payments.

Expenses can be split **equally** or by **custom (exact) amounts**. All four requirements were implemented and tested, and every group's balances add up to exactly Rs 0.00, which proves the accounting is correct.

2. Everything that was done, step by step

#	Step	What was done
1	**Understood the requirements**	Read the project brief and listed the 5 entities and 4 required queries.
2	**Designed the data model**	Decided which entities become collections and which are embedded inside other documents (members inside groups, participants inside expenses).
3	**Created the collections with validation**	Built 4 collections - users, groups, expenses, settlements - each with a $jsonSchema validator so MongoDB rejects bad data (e.g. a negative amount).
4	**Added indexes**	Created 7 indexes so look-ups by group, by payer, by participant and by email are fast. The email index is unique, so two accounts cannot share an email.
5	**Wrote helper functions (business rules)**	addUser, createGroup, joinGroup, addExpense (equal and exact split), recordSettlement and suggestSettlements. These enforce rules a schema cannot, such as 'shares must add up to the total'.
6	**Inserted sample data**	6 users, 2 groups (a flat in Koramangala and a Goa trip), 10 expenses and 2 earlier payments, chosen to show both split types and a user in two groups.
7	**Wrote the 4 queries**	Each is a MongoDB aggregation pipeline that runs inside the database.
8	**Tested on a real MongoDB server**	Ran the full script on MongoDB 8.3 and saved the output to expense_splitter_output.txt. Also checked that invalid data and duplicate emails are rejected.

#	Step	What was done
9	**Wrote the documentation**	A complete project guide and this summary, with ER diagrams, field tables, the pipelines and their results.
10	**Set it up on your Mac**	Installed Homebrew, MongoDB and mongosh, fixed the 'untrusted tap' error, ran the script successfully and opened the data in MongoDB Compass.

3. Software used and why

These are the tools needed to build and run the project on your computer.

Software	What it is	Why we used it
MongoDB Community Server (v8)	The database itself. Stores data as JSON-like documents in collections.	The project brief requires MongoDB. Its documents fit this problem well: an expense and its list of participants are saved and read together as one document.
mongosh (MongoDB Shell)	MongoDB's command-line tool. It connects to the database and runs JavaScript.	Runs the whole project with one command: mongosh expense_splitter.js. Also lets you type queries by hand during a demo or viva.
MongoDB Compass	MongoDB's free graphical app for browsing databases.	Shows the collections and documents visually, including the embedded arrays. Best tool for showing the examiner the data without typing commands.
Homebrew	The standard package manager for macOS.	Installs MongoDB and mongosh with a few commands and runs MongoDB as a background service, so there is no manual download or configuration.
Terminal (macOS)	The built-in command-line app on a Mac.	Where the Homebrew and mongosh commands are typed.
JavaScript	The programming language mongosh understands.	MongoDB's shell language. The schema, data, helper functions and queries are all written in one .js file, so the project is a single file to submit and run.

Why MongoDB instead of a SQL database (e.g. MySQL)?

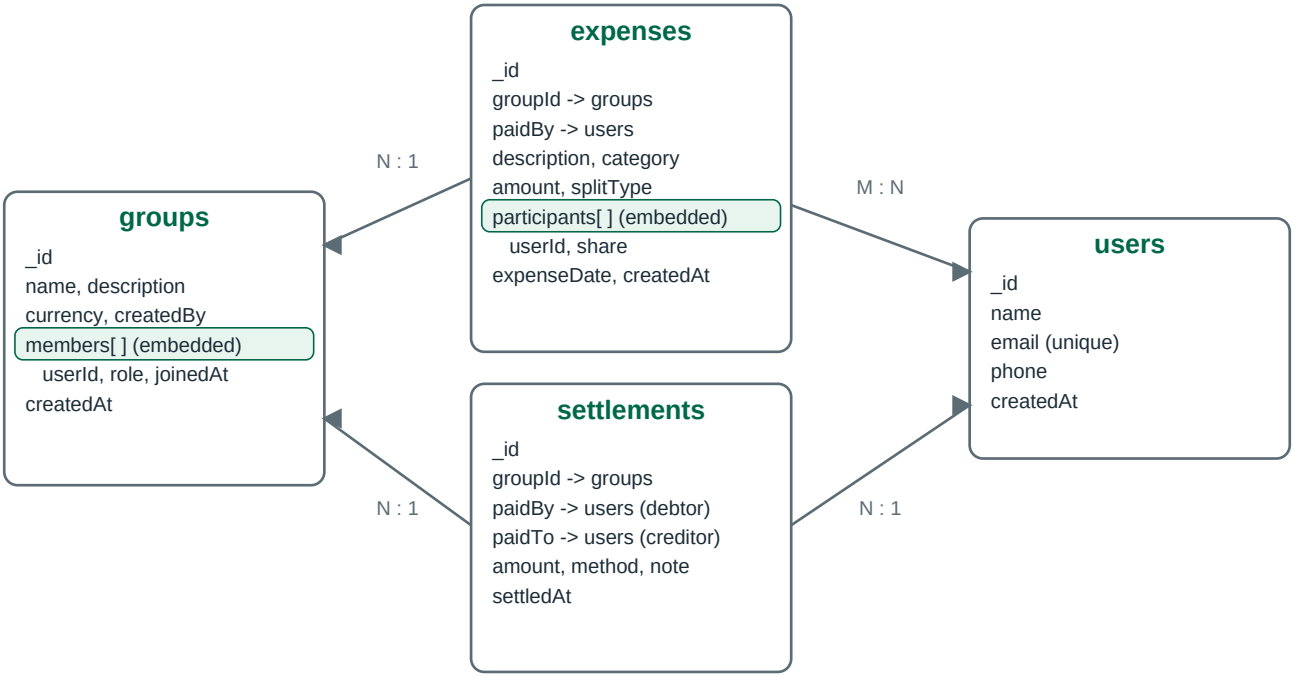
- **Natural fit for the data.** An expense has a variable-length list of participants. In SQL this needs a separate table and a JOIN on every read; in MongoDB it is an array inside the expense document.
- **Atomic writes.** Saving one document is all-or-nothing, so an expense can never be saved without its participants.
- **Validation is still possible.** $jsonSchema gives SQL-style rules (required fields, types, allowed values, minimums) inside MongoDB.
- **Powerful aggregation.** $group, $lookup, $unionWith and $facet calculate balances and reports inside the database.
- **Scales out.** If the app grew, data could be sharded (split across servers) by groupId.

Tools used only for testing (not needed on your Mac)

Before handing the project over, it was also tested on a separate Linux machine running MongoDB 8.3, with Node.js and mongosh installed through npm. Python (ReportLab) was used to create this PDF. None of these are needed to run the project yourself.

4. Database design

The 5 entities are stored in **4 collections**. Group Members are embedded inside **groups**, and Expense Participants are embedded inside **expenses**. Everything else is connected by ObjectId references (like foreign keys).



Shaded rows are embedded arrays stored inside the parent document. Arrows point from a reference field to the collection it refers to.

Embedding vs referencing

- **Embedded** (members, participants): small lists, always read with their parent, never needed on their own. One read gets everything.
- **Referenced** (users, groups, expenses, settlements): grow without limit and are queried on their own, so they have their own collections and are joined with $lookup when needed.
- **Balances are never stored.** They are calculated from expenses and settlements each time, so they can never go out of sync with the real records.

Indexes

Collection	Index	Purpose
users	email (unique)	Fast login look-up; blocks duplicate accounts
groups	members.userId	Find all groups a user belongs to
expenses	groupId + expenseDate	Queries 1 and 2: a group's expenses by date
expenses	paidBy	Query 3: expenses a user paid for
expenses	participants.userId	Query 3: expenses a user shares in
settlements	groupId + settledAt	Queries 2 and 4: a group's payments
settlements	paidBy + paidTo	Query 3: a user's payments

Business rules

- The payer and every participant must be members of the group.
- **Equal split** is calculated in paise so shares add up exactly: Rs 1,849 among 3 people becomes 616.34 + 616.33 + 616.33.
- **Exact split** is refused unless the shares add up to the expense amount.
- **Balance** = (amount paid for expenses + settlements paid) - (own share + settlements received). Positive means the member gets money back; negative means they owe.
- A settlement is only accepted from someone who owes to someone who is owed, never to oneself, and never more than what is outstanding.

5. The four key queries and their results

Sample data: **Flat 302 - Koramangala** (Aarav, Priya, Rohan, Sneha) and **Goa Trip 2026** (Karthik, Aarav, Meera, Rohan). Aarav and Rohan are in both groups. The figures below are the actual output from running the script.

Query 1 - All expenses and total spending of a group

How it works: filter the group's expenses ($match), add the payer's name ($lookup), then return the list, the total and a category breakdown in one pass ($facet).

Date	Description	Amount (Rs)	Paid by	Split
01 Oct	October rent	40,000.00	Aarav	Equal, 4
05 Oct	Electricity bill	2,350.00	Priya	Equal, 4
08 Oct	Groceries (Sneha away)	1,849.00	Rohan	Equal, 3
10 Oct	Wi-Fi recharge	999.00	Sneha	Equal, 4
12 Oct	Pizza night	1,600.00	Priya	Exact, 4
15 Oct	Deep cleaning service	1,200.00	Aarav	Equal, 4

Result: total spending Rs 47,998.00 across 6 expenses (average Rs 7,999.67).

Query 2 - How much each member owes or is owed

How it works: every expense is turned into ledger entries (+amount for the payer, -share for each participant); settlements are added with $unionWith; $group adds them up per person.

Member (Flat 302)	Paid	Own share	Net (Rs)	Status
Aarav Sharma	41,200.00	12,253.59	+23,946.41	Gets back
Sneha Reddy	999.00	11,437.25	-5,438.25	Owes
Priya Nair	3,950.00	12,053.58	-8,103.58	Owes
Rohan Mehta	1,849.00	12,253.58	-10,404.58	Owes

Aarav's net includes Rs 5,000 Sneha already paid him. The four nets add up to exactly 0.00. In the Goa trip, Karthik is owed Rs 8,085.00.

Query 3 - Expense history and contributions of a user

How it works: find expenses the user paid for or shares in, work out the user's own share with $filter, and return the history plus per-group totals with $facet.

Result for Aarav: involved in 9 expenses across 2 groups and paid for 3 of them. He paid Rs 47,200.00 in total against his own share of Rs 19,568.59. He also received Rs 5,000.00 from Sneha by UPI.

Query 4 - Unsettled balances and recording settlements

How it works: runs Query 2 for every group and keeps non-zero balances (8 before settling). It then suggests the fewest payments to clear a group, and records each payment with checks.

From	To	Amount (Rs)
Meera Iyer	Karthik Rao	5,515.00
Aarav Sharma	Karthik Rao	1,315.00
Rohan Mehta	Karthik Rao	1,255.00

After recording these 3 payments, every Goa Trip member shows 0.00 (settled up). An extra Rs 100 payment is correctly refused. Only the 4 Flat 302 balances remain unsettled.

6. Project files

File	What it contains
expense_splitter.js	The whole project in one script (548 lines): collections with validators, indexes, helper functions, sample data and the 4 queries.
expense_splitter_output.txt	The printed output of a full run, useful as an appendix or for screenshots.
SplitEase_Complete_Project_Guide.pdf	Full write-up: basics, why MongoDB, ER diagrams, field tables, indexes, business rules, each query's pipeline and results, the website, how to run, presentation script and viva Q&A.
This PDF	A plain-language summary of what was done and the software used.

Structure of expense_splitter.js

Section	Purpose
1. Collections	Drops and recreates the database; creates 4 collections with $jsonSchema rules
2. Indexes	Creates the 7 indexes
3. Helpers	Functions that add users, groups, expenses and settlements with rule checks
4. Sample data	Inserts the users, groups, expenses and payments; tests validation
5-8. Queries	Query 1, Query 2, Query 3 and Query 4 (4a-4d)

> Section 3 prints nothing, which is why the output jumps from section 2 to section 4.

7. How to run it on a Mac

One-time setup

Open **Terminal** (Command + Space, type Terminal, press Return) and run these one at a time:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
brew tap mongodb/brew
```

```
brew trust mongodb/brew
brew install mongodb-community mongosh
brew install --cask mongodb-compass
mkdir -p ~/Desktop/dbms-project
mv ~/Downloads/expense_splitter.js ~/Desktop/dbms-project/
```

Every time you run the project

```
brew services start mongodb-community
cd ~/Desktop/dbms-project && mongosh expense_splitter.js
```

The script resets the database each time, so it is always safe to run again. It ends with *Done. Database 'expense_splitter' is ready.*

Viewing the data in Compass

- Open **MongoDB Compass** and click the **127.0.0.1:27017** connection.
- Click **expense_splitter**, then any collection (users, groups, expenses, settlements).
- Expand **participants** in an expense, or **members** in a group, to show the embedded arrays.

Typing queries by hand (useful in a viva)

```
mongosh
use expense_splitter
db.users.find()
db.expenses.find({ category: "Food" })
db.settlements.find()
exit
```

When finished: `brew services stop mongodb-community`

8. Problems we hit and how they were fixed

Problem	Cause	Fix
Refusing to load formula ... from untrusted tap mongodb/brew	Newer Homebrew versions ask you to confirm you trust add-on sources.	Ran `brew trust mongodb/brew`. It is MongoDB's own official source.
brew: command not found	Homebrew was installed but Terminal did not know where it was.	Added it to ~/.zprofile with the shellenv command, then reopened Terminal.
ECONNREFUSED / connect failed	MongoDB server not running.	`brew services start mongodb-community`
No such file or directory	The script was not in the folder Terminal was in.	Move expense_splitter.js into ~/Desktop/dbms-project and cd there.
Database not visible in Compass	Compass was opened before the script ran.	Click the refresh icon next to Databases.

9. Conclusion and future scope

The project delivers a working MongoDB database for a shared-expense app. It stores users, groups, expenses with their participants, and settlements; supports equal and custom splits; and answers all four required queries. Because balances are calculated from the records rather than stored, they always add up to zero and can never fall out of sync.

Possible improvements

- Multi-document transactions so related writes commit together.
- Percentage and share-based splits.
- Multi-currency groups with exchange rates.
- Recurring expenses such as monthly rent.
- Receipt photos stored with GridFS, and reminders when someone owes too much.
- A web or mobile app built on top of this database.